

LAPORAN PRATIKUM 9
STRUKTUR DATA
“SEARCHING (BFS dan DFS)”

disusun Oleh:

Silva Nurifvan

2511533004

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data dengan judul “Searching (BFS dan DFS)” pada Pekan ke-9 ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan.

Praktikum ini membahas tentang algoritma pencarian pada struktur data graf, yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Kedua algoritma tersebut merupakan metode penting dalam proses penelusuran simpul pada graf yang banyak digunakan dalam berbagai bidang ilmu komputer, seperti pencarian jalur, kecerdasan buatan, jaringan komputer, dan pengolahan data.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai algoritma BFS dan DFS.

Padang, 10 Juni 2026

Penulis

PENDAULUAN

1.1 Latar Belakang

Struktur data graf merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek. Dalam implementasinya, sering kali diperlukan proses pencarian atau penelusuran terhadap simpul-simpul yang terdapat pada graf untuk menemukan informasi tertentu atau menentukan jalur yang dapat ditempuh. Untuk melakukan proses penelusuran tersebut, terdapat beberapa algoritma pencarian yang umum digunakan, di antaranya Breadth First Search (BFS) dan Depth First Search (DFS). BFS melakukan pencarian dengan menelusuri simpul berdasarkan tingkat kedekatan dari simpul awal, sedangkan DFS melakukan pencarian dengan menelusuri satu cabang hingga kedalaman tertentu sebelum berpindah ke cabang lainnya. Pemahaman terhadap kedua algoritma ini sangat penting karena menjadi dasar dalam penyelesaian berbagai permasalahan komputasi, seperti pencarian rute, traversal graf, analisis jaringan, dan berbagai aplikasi lainnya. Oleh karena itu, praktikum ini dilaksanakan agar mahasiswa dapat memahami konsep serta implementasi BFS dan DFS dalam pemrograman.

1.2 Tujuan Pratikum

1. Memahami konsep dasar pencarian (searching) pada struktur data graf.
2. Memahami cara kerja algoritma Breadth First Search (BFS).
3. Memahami cara kerja algoritma Depth First Search (DFS).
4. Mengimplementasikan algoritma BFS dan DFS menggunakan bahasa pemrograman Java.
5. Menganalisis hasil penelusuran simpul yang diperoleh dari kedua algoritma tersebut.
6. Membandingkan karakteristik dan perbedaan BFS dengan DFS dalam proses pencarian pada graf.

1.3 Manfaat Pratikum

1. Menambah pemahaman mengenai struktur data graf dan proses traversal graf.
2. Melatih kemampuan mahasiswa dalam mengimplementasikan algoritma BFS dan DFS ke dalam program Java.
3. Meningkatkan kemampuan berpikir logis dan analitis dalam menyelesaikan permasalahan pencarian data.
4. Memberikan pengalaman praktis dalam penggunaan struktur data seperti Queue pada BFS dan Stack/Recursion pada DFS.

BAB II

PEMBAHASAN

2.1 Pratikum “BTree_2511533004”

Kode Program

```
1 package pekan9_2511533004;
2
3 public class BTree_2511533004 {
4     private Node_2511533004 root_3004;
5     private Node_2511533004 currentNode_3004;
6     public BTree_2511533004() {
7         root_3004 = null;
8     }
9     public boolean search(int data_3004) {
10         return search(root_3004, data_3004);
11     }
12     private boolean search(Node_2511533004 node_3004, int data_3004) {
13         if (node_3004.getData() == data_3004) {
14             return true;
15         }
16         if (node_3004.getLeft() != null) {
17             if (search(node_3004.getLeft(), data_3004)) {
18                 return true;
19             }
20         }
21         if (node_3004.getRight() != null) {
22             if (search(node_3004.getRight(), data_3004)) {
23                 return true;
24             }
25         }
26         return false;
27     }
28     public void printInorder() {
29         root_3004.printInorder(root_3004);
30     }
31     public void printPreorder() {
32         root_3004.printPreorder(root_3004);
33     }
34     public void printPostOrder() {
35         root_3004.printPostOrder(root_3004);
36     }
37     public Node_2511533004 getRoot() {
38         return root_3004;
39     }
40     public boolean isEmpty() {
41         return root_3004 == null;
42     }
43 }
44
45 public int countNodes() {
46     return countNodes(root_3004);
47 }
48
49 private int countNodes(Node_2511533004 node) {
50     int count = 1;
51     if (node == null) {
52         return 0;
53     }
54     else {
55         count += countNodes(node.getLeft());
56         count += countNodes(node.getRight());
57         return count;
58     }
59 }
60
61 public void print() {
62     root_3004.print();
63 }
64
65 public Node_2511533004 getCurrent() {
66     return currentNode_3004;
67 }
68
69 public void setCurrent(Node_2511533004 node) {
70     this.currentNode_3004 = node;
71 }
72
73 public void setRoot_3004(Node_2511533004 root) {
74     this.root_3004 = root;
75 }
```

PENJELASAN

1. Deskripsi Umum

Kode ini merupakan implementasi struktur data **Pohon Biner (Binary Tree)** dalam bahasa pemrograman Java. Kelas utama bernama BTree_2511533004 berfungsi mengelola seluruh operasi yang bisa dilakukan pada pohon biner, seperti pencarian data, penelusuran (traversal), penghitungan jumlah simpul, serta pengelolaan akar dan simpul saat ini. Kode ini terhubung dengan kelas pendukung bernama Node_2511533004 yang merepresentasikan setiap elemen atau simpul di dalam pohon.

Catatan: Angka di belakang nama kelas dan variabel (3004 / 2511533004) merupakan penanda khusus, biasanya berupa kode unik atau nomor identitas penulis program.

2. Komponen Utama

A. Atribut / Variabel Anggota

java

Run

```
private Node_2511533004 root_3004;  
private Node_2511533004 currentNode_3004;
```

- **root_3004:** Merupakan variabel yang menyimpan alamat simpul **akar** dari pohon biner. Akar adalah simpul paling atas dalam struktur pohon.
- **currentNode_3004:** Variabel bantu yang menyimpan alamat simpul yang sedang aktif atau sedang diproses dalam operasi tertentu.

B. Konstruktor

java

Run

```
public BTree_2511533004() {  
    root_3004 = null;  
}
```

Konstruktor adalah metode yang pertama kali dijalankan saat objek pohon dibuat. Pada kode ini, konstruktor menginisialisasi pohon dalam keadaan kosong dengan mengatur nilai akar (root_3004) menjadi null.

3. Penjelasan Metode (Fungsi)

Metode Pencarian (search)

java

Run

```
public boolean search(int data_3004)  
private boolean search(Node_2511533004 node_3004, int data_3004)
```

- **Fungsi:** Mencari apakah suatu nilai data tertentu ada di dalam pohon biner.
- **Cara Kerja:**
 1. Metode publik search(int data) dipanggil pengguna, lalu memanggil metode pribadi search dengan mengirimkan simpul akar sebagai titik awal pencarian.
 2. Jika simpul yang diperiksa berisi data yang dicari, kembalikan nilai true (ditemukan).
 3. Jika tidak ditemukan di simpul tersebut, lakukan pencarian berulang kali secara rekursif ke **cabang kiri**, lalu ke **cabang kanan**.

4. Jika seluruh cabang sudah diperiksa dan tidak ditemukan, kembalikan nilai false (tidak ditemukan).

Konsep: Menggunakan algoritma pencarian *Depth-First Search* (jelajah mendalam).

Metode Penelusuran / Traversal

Digunakan untuk menampilkan seluruh data yang ada di dalam pohon dengan urutan tertentu:

1. **printInorder()**

- Menampilkan data dengan urutan: **Anak Kiri → Induk → Anak Kanan.**
- Hasilnya biasanya akan menampilkan nilai secara berurutan dari kecil ke besar pada pohon biner teratur.

2. **printPreOrder()**

- Menampilkan data dengan urutan: **Induk → Anak Kiri → Anak Kanan.**
- Berguna untuk menyalin struktur pohon atau membuat ekspresi awalan.

3. **printPostOrder()**

- Menampilkan data dengan urutan: **Anak Kiri → Anak Kanan → Induk.**
- Berguna untuk menghapus pohon atau menghitung nilai ekspresi.

Ketiga metode ini memanggil metode dengan nama yang sama yang terdapat di dalam kelas Node, yang akan berjalan secara rekursif menelusuri setiap cabang.

Metode Penghitungan Jumlah Simpul (countNodes)

java

Run

```
public int countNodes()
```

```
private int countNodes(Node_2511533004 node)
```

- **Fungsi:** Menghitung berapa banyak simpul/elemen yang ada di dalam pohon.
- **Cara Kerja:**
 1. Jika simpul yang diperiksa bernilai null (kosong), maka jumlahnya adalah 0.
 2. Jika ada isinya, hitung simpul tersebut sebagai 1, lalu tambahkan jumlah simpul dari cabang kiri dan cabang kanan secara rekursif.
 3. Hasil penjumlahan dikembalikan sebagai jumlah total simpul.

Metode Pengecekan Kondisi (isEmpty)

java

Run

```
public boolean isEmpty() {
    return root_3004 == null;
}
```

- **Fungsi:** Memeriksa apakah pohon dalam keadaan kosong atau tidak.
- **Hasil:** Mengembalikan nilai true jika akar bernilai null (kosong), dan false jika sudah ada isinya.

Metode Pengaturan & Pengambilan Data (Getter & Setter)

Berfungsi sebagai antarmuka untuk mengakses atau mengubah nilai atribut dari luar kelas:

1. **getRoot():** Mengembalikan objek simpul akar pohon saat ini.
2. **getCurrent():** Mengembalikan objek simpul yang sedang aktif/diproses.
3. **setCurrent(Node node):** Mengatur ulang simpul mana yang ditetapkan sebagai simpul aktif.
4. **setRoot_3004(Node root):** Mengatur atau mengganti simpul akar pohon dengan simpul baru.
5. **print():** Memanggil fungsi print() dari simpul akar untuk menampilkan seluruh isi pohon (implementasi tampilan ada di kelas Node).

4. Kesimpulan

Kelas BTree_2511533004 ini berperan sebagai **pengelola utama** struktur pohon biner. Kode ini menerapkan prinsip pemrograman berorientasi objek dengan menyembunyikan detail data (enkapsulasi) melalui penggunaan atribut private, dan menyediakan metode publik untuk berinteraksi dengan struktur data tersebut.

Fitur utama yang telah diimplementasikan meliputi:

- Pembuatan pohon kosong.
- Pencarian data.
- Tiga jenis penelusuran (Inorder, Preorder, Postorder).
- Penghitungan jumlah elemen.
- Pengelolaan simpul akar dan simpul aktif.

Program ini merupakan dasar yang lengkap untuk memahami dan mengembangkan struktur data pohon biner lebih lanjut, seperti penambahan fitur penyisipan data, penghapusan data, atau penyeimbangan pohon.

2.2 Pratikum “Node_2511533004”

Kode Program

```
1 package pekab_2511533004;
2
3 public class Node_2511533004 {
4     int data_3004;
5     Node_2511533004 left_3004;
6     Node_2511533004 right_3004;
7
8     public Node_2511533004(int data) {
9         this.data_3004 = data;
10        this.left_3004 = null;
11        this.right_3004 = null;
12    }
13
14    public void setLeft(Node_2511533004 node) {
15        if (left_3004 == null) {
16            left_3004 = node;
17        }
18    }
19
20    public void setRight(Node_2511533004 node) {
21        if (right_3004 == null) {
22            right_3004 = node;
23        }
24    }
25
26    public Node_2511533004 getLeft() {
27        return left_3004;
28    }
29
30    public Node_2511533004 getRight() {
31        return right_3004;
32    }
33
34    public int getData() {
35        return data_3004;
36    }
37
38    public void setData(int data) {
39        this.data_3004 = data;
40    }
41
42    void printPreOrder(Node_2511533004 node) {
43        if (node == null) return;
44        System.out.print(node.data_3004 + " ");
45        printPreOrder(node.left_3004);
46        printPreOrder(node.right_3004);
47    }
48
49    void printInOrder(Node_2511533004 node) {
50        if (node == null) return;
51        printInOrder(node.left_3004);
52        printInOrder(node.right_3004);
53        System.out.print(node.data_3004 + " ");
54    }
55
56    void printPostOrder(Node_2511533004 node) {
57        if (node == null) return;
58        printInOrder(node.left_3004);
59        System.out.print(node.data_3004 + " ");
60        printInOrder(node.right_3004);
61    }
62
63    public String prelev() {
64        return print("", true, "");
65    }
66
67    public String print(String prefix, boolean isTail, String sb) {
68        if (right_3004 != null) {
69            right_3004.print(prefix + (isTail ? " " : "  "), false, sb);
70        }
71
72        System.out.print(prefix + (isTail ? " /--- " : "/--- ") + data_3004);
73
74        if (left_3004 != null) {
75            left_3004.print(prefix + (isTail ? " " : "  "), true, sb);
76        }
77
78        return sb;
79    }
80 }
```

PENJELASAN

1. Deskripsi Umum

Kode ini adalah implementasi kelas **Node_2511533004**, yang berfungsi sebagai komponen dasar atau elemen pembentuk struktur data **Pohon Biner (Binary Tree)**. Setiap objek dari kelas ini merepresentasikan satu simpul yang menyimpan nilai data serta alamat ke dua cabang turunannya, yaitu anak kiri dan anak kanan. Kelas ini menjadi pendukung utama bagi kelas **BTree_2511533004** yang telah dijelaskan sebelumnya, karena tanpa kelas ini, struktur pohon tidak dapat dibentuk.

Angka 2511533004 dan akhiran _3004 pada nama variabel merupakan penanda identitas, biasanya berupa nomor mahasiswa atau kode unik pembuat program.

2. Atribut / Variabel Anggota

java

Run

```
int data_3004;
```

```
Node_2511533004 left_3004;
```

```
Node_2511533004 right_3004;
```

- **data_3004:** Variabel bertipe bilangan bulat (int), berfungsi menyimpan nilai atau informasi yang ada di dalam simpul tersebut.
- **left_3004:** Referensi/alamat yang menunjuk ke objek simpul lain sebagai **anak kiri** dari simpul saat ini. Bernilai null jika tidak memiliki anak kiri.
- **right_3004:** Referensi/alamat yang menunjuk ke objek simpul lain sebagai **anak kanan** dari simpul saat ini. Bernilai null jika tidak memiliki anak kanan.

Karena atribut tidak menggunakan kata kunci private, artinya atribut ini bisa diakses langsung oleh kelas lain dalam paket yang sama.

3. Konstruktor

java

Run

```
public Node_2511533004(int data) {  
    this.data_3004 = data;  
    this.left_3004 = null;  
    this.right_3004 = null;  
}
```

Metode ini dijalankan saat objek simpul baru dibuat.

- Menerima satu parameter berupa nilai data yang akan disimpan.
- Nilai tersebut langsung dimasukkan ke dalam atribut data_3004.
- Secara otomatis mengatur cabang kiri dan kanan menjadi null, yang berarti saat pertama kali dibuat, simpul belum memiliki anak.

4. Metode Pengaturan dan Pengambilan Nilai (Getter & Setter)

```
setLeft(Node_2511533004 node)
```

- **Fungsi:** Menambahkan atau mengatur simpul anak di sebelah kiri.
- **Khusus:** Memiliki syarat, nilai hanya akan dimasukkan jika cabang kiri saat ini masih kosong (null). Jika sudah ada isinya, perubahan tidak akan dilakukan.

```
setRight(Node_2511533004 node)
```

- **Fungsi:** Menambahkan atau mengatur simpul anak di sebelah kanan.
- **Khusus:** Sama seperti metode setLeft, nilai baru hanya dimasukkan jika cabang kanan masih kosong.

getLeft()

- Mengembalikan objek simpul yang menjadi anak kiri dari simpul saat ini. Digunakan untuk proses penelusuran ke cabang kiri.

getRight()

- Mengembalikan objek simpul yang menjadi anak kanan dari simpul saat ini. Digunakan untuk proses penelusuran ke cabang kanan.

getData()

- Mengembalikan nilai data yang tersimpan di dalam simpul.

setData(int data)

- Mengubah atau memperbarui nilai data yang tersimpan di dalam simpul dengan nilai baru.

5. Metode Penelusuran (Traversal)

Ketiga metode di bawah ini bekerja secara **rekursif** (memanggil dirinya sendiri) untuk menelusuri seluruh isi pohon yang berada di bawah simpul tertentu. Proses ini berhenti jika menemukan simpul kosong (null).

◇ printPreOrder(Node_2511533004 node)

- **Urutan: Induk → Kiri → Kanan**
- Cara kerja: Tampilkan data simpul sekarang, lalu telusuri seluruh cabang kiri, terakhir telusuri seluruh cabang kanan.

◇ printPostOrder(Node_2511533004 node)

- **Urutan: Kiri → Kanan → Induk**
- Cara kerja: Telusuri seluruh cabang kiri, telusuri seluruh cabang kanan, baru tampilkan data simpul sekarang.

◇ printInorder(Node_2511533004 node)

- **Urutan: Kiri → Induk → Kanan**
- Cara kerja: Telusuri seluruh cabang kiri, tampilkan data simpul sekarang, lalu telusuri seluruh cabang kanan.
- *Pada pohon biner teratur, urutan ini akan menampilkan data dari nilai terkecil hingga terbesar.*

Hasil tampilan menggunakan System.out.print, sehingga semua nilai akan dicetak dalam satu baris berurutan.

6. Metode Tampilan Struktur Pohon (print())

Bagian ini merupakan fitur tambahan yang canggih, berfungsi menampilkan struktur pohon ke layar dalam bentuk **diagram teks yang dapat dibaca**, lengkap dengan garis penghubung antar simpul.

```
public String print()
```

- Merupakan metode awal yang dipanggil. Berfungsi memulai proses pencetakan dengan parameter standar, lalu mengembalikan hasilnya berupa teks.

```
private String print(String prefix, boolean isTail, String sb)
```

- Ini adalah metode utama yang bekerja secara rekursif membangun tampilan pohon.
 - prefix: Mengatur jarak dan garis penghubung di sebelah kiri simpul agar susunan tampilan rapi.
 - isTail: Penanda apakah simpul ini adalah simpul terakhir pada tingkat tersebut, menentukan simbol garis yang digunakan (\-- atau +--).
 - sb: Variabel penampung teks hasil cetakan.
- **Alur kerja:**
 1. Memproses dan mencetak cabang kanan terlebih dahulu.
 2. Mencetak garis penghubung dan nilai data simpul saat ini.
 3. Memproses dan mencetak cabang kiri.
- Hasil akhirnya adalah teks yang membentuk diagram pohon yang mudah dimengerti secara visual.

7. Kesimpulan

Kelas `Node_2511533004` adalah komponen paling dasar dalam pembentukan pohon biner. Kelas ini bertugas menyimpan data dan koneksi antar simpul, serta menyediakan fungsi untuk menelusuri dan menampilkan isi pohon.

Fitur utama yang ada:

1. Penyimpanan nilai data dan alamat anak kiri-kanan.
2. Pengaturan koneksi antar simpul dengan aturan keamanan (tidak bisa menimpa anak yang sudah ada).
3. Tiga jenis penelusuran standar: Preorder, Inorder, Postorder.
4. Fitur tampilan visual struktur pohon dalam bentuk diagram teks.

Tanpa kelas ini, kelas `BTree` yang mengelola pohon tidak akan berfungsi, karena kelas `BTree` hanya bertugas mengatur alur kerja, sedangkan kelas `Node` yang memegang data sebenarnya.

2.3 Pratikum “BTreeDriver_2511533004”

Kode Program

```
1 package pekan9_2511533004;
2
3 public class BTreeDriver_2511533004 {
4     public static void main(String[] args) {
5         //Membuat pohon
6         BTree_2511533004 tree_3004 = new BTree_2511533004();
7         System.out.println("Jumlah simpul awal pohon: ");
8         System.out.println(tree_3004.countNodes());
9
10        //Menambahkan simpul data 1
11        Node_2511533004 root = new Node_2511533004(1);
12        //menjadikan simpul 1 sebagai root
13        tree_3004.setRoot_3004(root);
14        System.out.println("Jumlah simpul jika hanya ada root?");
15        System.out.println(tree_3004.countNodes());
16
17        Node_2511533004 node2 = new Node_2511533004(2);
18        Node_2511533004 node3 = new Node_2511533004(3);
19        Node_2511533004 node4 = new Node_2511533004(4);
20        Node_2511533004 node5 = new Node_2511533004(5);
21        Node_2511533004 node6 = new Node_2511533004(6);
22        Node_2511533004 node7 = new Node_2511533004(7);
23        Node_2511533004 node8 = new Node_2511533004(8);
24        Node_2511533004 node9 = new Node_2511533004(9);
25
26        root.setLeft(node2);
27        node2.setLeft(node4);
28        node2.setRight(node5);
29        node4.setRight(node8);
30        root.setRight(node3);
31        node3.setLeft(node6);
32        node3.setRight(node7);
33        node6.setLeft(node9);
34
35        //Set root
36        tree_3004.setCurrent(tree_3004.getRoot());
37        System.out.println("\nMenampilkan simpul terakhir: ");
38        System.out.println(tree_3004.getCurrent().getData());
39
40        System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
41        System.out.println(tree_3004.countNodes());
42
43        System.out.println("\nInOrder: ");
44        tree_3004.printInOrder();
45
46        System.out.println("\nPreorder: ");
47        tree_3004.printPreOrder();
48
49        System.out.println("\nPostorder: ");
50        tree_3004.printPostOrder();
51
52        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
53        tree_3004.print();
54    }
55 }
```

PENJELASAN

1. Deskripsi Umum

Kelas **BtreeDriver_2511533004** adalah kelas utama (program utama) yang berfungsi sebagai penguji dan penerapan dari kelas BTree_2511533004 dan Node_2511533004 yang telah dijelaskan sebelumnya. Di dalam kelas ini terdapat metode main, tempat eksekusi program dimulai. Kode ini bertujuan untuk mendemonstrasikan cara kerja struktur data pohon biner, mulai dari pembuatan objek, penambahan simpul, penyusunan hubungan antar simpul, hingga menampilkan hasilnya melalui berbagai metode penelusuran dan tampilan visual.

2. Penjelasan Kode Baris per Bagian

A. Pembuatan Objek Pohon

java

Run

```
BTree_2511533004 tree_3004 = new BTree_2511533004();
```

```
System.out.print("Jumlah Simpul awal pohon: ");
```

```
System.out.println(tree_3004.countNodes());
```

- Membuat objek baru bernama tree_3004 dari kelas BTree. Saat baru dibuat, pohon masih kosong.
- Menampilkan jumlah simpul awal, yang hasilnya akan **0**, karena belum ada data yang dimasukkan.

B. Pembuatan dan Penetapan Simpul Akar

java

Run

```
Node_2511533004 root = new Node_2511533004(1);
```

```
tree_3004.setRoot_3004(root);
```

```
System.out.println("Jumlah simpul jika hanya ada root");
```

```
System.out.println(tree_3004.countNodes());
```

- Membuat simpul baru dengan nilai data **1**, yang ditetapkan sebagai **akar (puncak)** pohon.
- Mengatur objek tree agar mengenali simpul ini sebagai akar pohon menggunakan metode setRoot.
- Menampilkan jumlah simpul, sekarang hasilnya **1**, karena sudah ada simpul akar.

C. Pembuatan Simpul Lainnya

java

Run

```
Node_2511533004 node2 = new Node_2511533004(2);
```

```
Node_2511533004 node3 = new Node_2511533004(3);
```

```
Node_2511533004 node4 = new Node_2511533004(4);
```

```
Node_2511533004 node5 = new Node_2511533004(5);
```

```
Node_2511533004 node6 = new Node_2511533004(6);
```

```
Node_2511533004 node7 = new Node_2511533004(7);
```

```
Node_2511533004 node8 = new Node_2511533004(8);
```

```
Node_2511533004 node9 = new Node_2511533004(9);
```

- Membuat 8 objek simpul baru dengan nilai berturut-turut dari 2 hingga 9. Saat ini, simpul-simpul ini berdiri sendiri dan belum terhubung satu sama lain.

D. Penyusunan Hubungan Antar Simpul (Pembentukan Struktur Pohon)

java

Run

```
root.setLeft(node2);
```

```
node2.setLeft(node4);
node2.setRight(node5);
node4.setRight(node8);
```

```
root.setRight(node3);
node3.setLeft(node6);
node3.setRight(node7);
node6.setLeft(node9);
```

Bagian ini adalah inti pembentukan struktur pohon, yaitu mengatur siapa menjadi anak kiri dan anak kanan dari setiap simpul induk:

1. **Akar (1):**
 - Anak kiri = Simpul 2
 - Anak kanan = Simpul 3
2. **Simpul 2:**
 - Anak kiri = Simpul 4
 - Anak kanan = Simpul 5
3. **Simpul 4:**
 - Anak kanan = Simpul 8
(Anak kiri kosong)
4. **Simpul 3:**
 - Anak kiri = Simpul 6
 - Anak kanan = Simpul 7
5. **Simpul 6:**
 - Anak kiri = Simpul 9
(Anak kanan kosong)

Struktur pohon yang terbentuk secara skema:

plaintext

```
  1
 /  \
2    3
/\   /\
4 5 6 7
 \  /
  8 9
```

Perhatian: Berdasarkan logika metode setLeft dan setRight di kelas Node, koneksi hanya berhasil jika cabang masih kosong. Pada urutan kode ini, semua koneksi berjalan lancar karena dilakukan sekali saja.

E. Pengaturan Simpul Aktif

java

Run

```
tree_3004.setCurrent(tree_3004.getRoot());
```

```
System.out.println("Menampilkan simpul terakhir: ");
```

```
System.out.println(tree_3004.getCurrent().getData());
```

- Mengatur simpul yang sedang diproses (currentNode) menjadi sama dengan akar pohon.
- Menampilkan nilai data dari simpul aktif tersebut, hasilnya adalah **1**.

F. Penghitungan Jumlah Simpul Akhir

java

Run

```
System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
```

```
System.out.println(tree_3004.countNodes());
```

- Menghitung total seluruh simpul yang sudah terhubung dalam pohon.
- Berdasarkan struktur di atas, jumlah total simpul adalah **9**.

G. Penelusuran Pohon (Traversal)

java

Run

```
System.out.println("InOrder:");
```

```
tree_3004.printInorder();
```

```
System.out.println("\nPreorder: ");
```

```
tree_3004.printPreOrder();
```

```
System.out.println("\nPostorder: ");
```

```
tree_3004.printPostOrder();
```

Bagian ini memanggil ketiga metode penelusuran untuk melihat urutan data:

1. **InOrder (Kiri - Induk - Kanan)**

Urutan: 4 8 2 5 1 9 6 3 7

(Dimulai dari cabang kiri paling dalam, naik ke induk, lalu ke kanan, kemudian ke cabang kanan utama)

2. **PreOrder (Induk - Kiri - Kanan)**

Urutan: 1 2 4 8 5 3 6 9 7

(Dimulai dari akar, lalu jelajah ke kiri sampai habis, baru ke kanan)

3. **Postorder (Kiri - Kanan - Induk)**

Urutan: 8 4 5 2 9 6 7 3 1

(Dimulai dari daun paling kiri, ke kanan, baru ke induk, sampai akar terakhir)

H. Tampilan Visual Pohon

java

Run

```
System.out.println("\nMenampilkan simpul dalam bentuk pohon");
```

```
tree_3004.print();
```

- Memanggil metode print() yang ada di dalam kelas Node.
- Menampilkan struktur pohon dalam bentuk diagram teks lengkap dengan garis penghubung, sehingga bentuk hirarki pohon dapat dilihat dengan jelas di layar konsol.

3. Hasil Keluaran Program

Berikut adalah gambaran hasil yang akan muncul saat program dijalankan:

text

Jumlah Simpul awal pohon: 0

Jumlah simpul jika hanya ada root

1

Menampilkan simpul terakhir:

1

Jumlah simpul setelah simpul 7 ditambahkan

9

InOrder:

4 8 2 5 1 9 6 3 7

Preorder:

1 2 4 8 5 3 6 9 7

Postorder:

8 4 5 2 9 6 7 3 1

Menampilkan simpul dalam bentuk pohon

```
      └─ 7
└─ 3
  └─ 6
  └─ 9
1
└─ 5
  └─ 2
    └─ 4
      └─ 8
```


4. Kesimpulan

Kelas BtreeDriver_2511533004 berfungsi sebagai **program uji coba lengkap**. Kode ini membuktikan bahwa implementasi kelas Node dan BTree sudah berjalan sesuai logika struktur data pohon biner.

Melalui program ini, terbukti bahwa:

1. Objek pohon dapat dibuat dan dikelola.
2. Simpul dapat dihubungkan satu sama lain membentuk struktur bertingkat.
3. Penghitungan jumlah simpul berjalan akurat.
4. Tiga metode penelusuran (Inorder, Preorder, Postorder) bekerja sesuai urutan algoritmanya.
5. Struktur pohon dapat ditampilkan dalam bentuk diagram yang mudah dipahami.

Program ini adalah contoh penerapan nyata konsep pemrograman berorientasi objek dan struktur data pohon biner dalam bahasa Java.

2.4 Pratikum “GraphTraversal_2511533004”

Kode Program

```
1 package pekan0_2511533004;
2 import java.util.*;
3
4 public class GraphTraversal_2511533004 {
5     // Inisialisasi graph pada variabel
6     private Map<String, List<String>> graph_3004 = new HashMap<>();
7
8     // Inisialisasi graph pada nama metode & parameter
9     public void addEdge_3004(String node1_3004, String node2_3004) {
10         graph_3004.putIfAbsent(node1_3004, new ArrayList<>());
11         graph_3004.putIfAbsent(node2_3004, new ArrayList<>());
12         graph_3004.get(node1_3004).add(node2_3004);
13         graph_3004.get(node2_3004).add(node1_3004);
14     }
15
16     // Inisialisasi graph pada nama metode
17     public void printGraph_3004() {
18         System.out.println("Graf Dual (Adjacency List):");
19         for (String node_3004 : graph_3004.keySet()) {
20             System.out.print(node_3004 + " -> ");
21             List<String> neighbors_3004 = graph_3004.get(node_3004);
22             System.out.println(String.join(" ", neighbors_3004));
23         }
24         System.out.println();
25     }
26
27     // Inisialisasi graph pada nama metode & parameter
28     public void dfs_3004(String start_3004) {
29         Set<String> visited_3004 = new HashSet<>();
30         System.out.println("Penelusuran DFS:");
31         dfsHelper_3004(start_3004, visited_3004);
32         System.out.println();
33     }
34
35     private void dfsHelper_3004(String current_3004, Set<String> visited_3004) {
36         if (visited_3004.contains(current_3004)) return;
37         visited_3004.add(current_3004);
38         System.out.print(current_3004 + " ");
39     }
40 }
```

```

39     for (String neighbor_3004 : graph_3004.getDefault(current_3004, new ArrayList<>())) {
40         dfsHelper_3004(neighbor_3004, visited_3004);
41     }
42 }
43
44 // lakukan akhiran_3004 pada nama metode & parameter
45 public void bfs_3004(String start_3004) {
46     Set<String> visited_3004 = new HashSet<>();
47     Queue<String> queue_3004 = new LinkedList<>();
48     queue_3004.add(start_3004);
49     visited_3004.add(start_3004);
50
51     System.out.println("Penelusuran BFS:");
52     while (!queue_3004.isEmpty()) {
53         String current_3004 = queue_3004.poll();
54         System.out.print(current_3004 + " ");
55         for (String neighbor_3004 : graph_3004.getDefault(current_3004, new ArrayList<>())) {
56             if (!visited_3004.contains(neighbor_3004)) {
57                 queue_3004.add(neighbor_3004);
58                 visited_3004.add(neighbor_3004);
59             }
60         }
61     }
62     System.out.println();
63 }
64
65 public static void main(String[] args) {
66     GraphTraversal_2511533004 graph_3004 = new GraphTraversal_2511533004();
67
68     // DI BAGIAN INI TETAP POLOS, TANPA AKHIRAN_3004
69     graph_3004.addEdge_3004("A", "B");
70     graph_3004.addEdge_3004("A", "C");
71     graph_3004.addEdge_3004("B", "D");
72     graph_3004.addEdge_3004("B", "E");
73
74     System.out.println("Graf Awal adalah:");
75     graph_3004.printGraph_3004();
76
77     // JALURAN DIBERIKAN (DILAKUKAN AKHIRAN_3004)
78     graph_3004.dfs_3004("A");
79     graph_3004.bfs_3004("A");
80 }

```

PENJELASAN

1. Deskripsi Umum

Kode ini merupakan implementasi struktur data **Graf** beserta algoritma penelusurannya, yaitu **DFS (Depth-First Search)** dan **BFS (Breadth-First Search)** dalam bahasa Java. Kelas `GraphTraversal_2511533004` menggunakan pendekatan **Daftar Ketetanggaan (Adjacency List)** untuk menyimpan hubungan antar simpul. Kode ini mendemonstrasikan bagaimana cara membangun graf, menampilkan strukturnya, serta melakukan penelusuran mulai dari satu titik awal ke seluruh simpul yang terhubung.

Angka 2511533004 dan akhiran `_3004` adalah penanda identitas pembuat program.

2. Komponen Utama

A. Atribut / Variabel

java

Run

```
private Map<String, List<String>> graph_3004 = new HashMap<>();
```

- **graph_3004:** Variabel bertipe Map (Peta) yang berfungsi menyimpan struktur graf.
 - **Kunci (Key):** Bertipe String, mewakili nama simpul (misal: "A", "B", "C").

- **Nilai (Value):** Daftar (List) dari String, berisi kumpulan simpul lain yang terhubung langsung dengan kunci tersebut.
- Menggunakan HashMap agar akses data cepat dan simpul tersimpan secara unik.

Import java.util. digunakan agar bisa menggunakan koleksi data seperti Map, List, ArrayList, HashSet, dan LinkedList.*

3. Penjelasan Metode

`addEdge_3004(String node1, String node2)`

Fungsi: Menambahkan hubungan/garis penghubung antara dua simpul ke dalam graf.

Cara Kerja:

1. Memastikan simpul node1 ada di dalam peta; jika belum ada, buat daftar kosong untuk simpul tersebut.
2. Memastikan simpul node2 ada di dalam peta; jika belum ada, buat daftar kosong untuk simpul tersebut.
3. Menambahkan node2 ke dalam daftar tetangga node1.
4. Menambahkan node1 ke dalam daftar tetangga node2.

Keterangan: Karena hubungan dimasukkan ke dua arah, ini berarti program ini membangun **Graf Tidak Berarah** (hubungan timbal balik).

`printGraph_3004()`

Fungsi: Menampilkan seluruh isi graf dalam bentuk Daftar Ketetanggaan agar mudah dibaca.

Cara Kerja:

1. Melakukan pengulangan untuk setiap simpul yang ada di dalam peta.
2. Mencetak nama simpul, diikuti tanda panah ->.
3. Mencetak semua simpul yang terhubung dengannya, dipisahkan dengan tanda koma.

Contoh Hasil: A -> B, C artinya simpul A terhubung ke B dan C.

`dfs_3004(String start) & dfsHelper_3004(...)`

Fungsi: Melakukan penelusuran **Mendalam (DFS)**. Algoritma ini menjelajahi satu cabang/jalur sampai ujungnya terlebih dahulu, baru kembali dan menjelajahi cabang lain.

Cara Kerja:

1. **dfs_3004:** Metode awal. Membuat objek visited bertipe HashSet untuk mencatat simpul yang sudah dikunjungi agar tidak terulang. Memanggil metode pembantu dengan titik awal.

2. **dfsHelper_3004**: Metode rekursif (memanggil dirinya sendiri):

- Jika simpul sekarang sudah dikunjungi, hentikan proses ini (return).
- Tandai simpul sebagai sudah dikunjungi.
- Cetak nama simpul tersebut.
- Lakukan pengulangan ke semua simpul tetangga, lalu panggil metode ini kembali untuk setiap tetangganya.

Konsep Utama: Menggunakan mekanisme tumpukan (secara implisit lewat pemanggilan fungsi) untuk masuk sedalam mungkin ke dalam graf.

bfs_3004(String start)

Fungsi: Melakukan penelusuran **Melebar (BFS)**. Algoritma ini menjelajahi semua simpul yang berjarak 1 langkah (tetangga langsung) terlebih dahulu, baru bergerak ke simpul yang berjarak 2 langkah, dan seterusnya.

Cara Kerja:

1. Membuat visited untuk mencatat simpul yang sudah dilewati.
2. Membuat antrean (queue) menggunakan LinkedList untuk mengatur urutan simpul yang akan diproses.
3. Memasukkan simpul awal ke dalam antrean dan tandai sebagai sudah dikunjungi.
4. **Perulangan:**
 - Mengambil dan mengeluarkan simpul dari depan antrean (poll), lalu cetak.
 - Cek semua tetangga dari simpul tersebut.
 - Jika tetangga belum dikunjungi, masukkan ke antrean dan tandai sudah dikunjungi.
5. Proses berlanjut sampai antrean kosong.

Konsep Utama: Menggunakan struktur data **Antrean** (Masuk belakang, Keluar depan) untuk menjaga urutan penelusuran berjenjang/bertingkat.

main(String[] args)

Fungsi: Metode utama tempat program dijalankan dan diuji.

Langkah Eksekusi:

1. Membuat objek graf baru bernama graph_3004.
2. Membangun struktur graf dengan menambahkan hubungan antar simpul:
plaintext
A <-> B
A <-> C
B <-> D

B <-> E

Bentuk graf:

plaintext

C

/

A - B - D

\

E

3. Menampilkan struktur graf yang telah dibuat.
4. Menjalankan penelusuran **DFS** dimulai dari simpul "A".
5. Menjalankan penelusuran **BFS** dimulai dari simpul "A".

4. Hasil Keluaran Program

Berikut adalah urutan hasil yang akan muncul saat program dijalankan:

text

Graf Awal adalah:

Graf Awal (Adjacency List):

A -> B, C

B -> A, D, E

C -> A

D -> B

E -> B

Penelusuran DFS:

A B D E C

Penelusuran BFS:

A B C D E

Penjelasan Hasil:

- **DFS:** Mulai dari A → ke B → ke D (habis) → kembali ke B → ke E (habis) → kembali ke A → ke C (selesai).
- **BFS:** Mulai dari A → kunjungi semua tetangga langsung: B, C → lanjut ke tetangga B: D, E.

5. Kesimpulan

Kelas GraphTraversal_2511533004 ini merupakan implementasi lengkap konsep dasar graf. Program ini mampu:

1. Membangun hubungan antar simpul secara dinamis.

2. Menyimpan data graf secara efisien menggunakan daftar ketetanggaan.
3. Menampilkan struktur graf agar mudah dipahami.
4. Melakukan penelusuran mendalam (DFS) yang cocok untuk mencari jalur atau siklus.
5. Melakukan penelusuran melebar (BFS) yang cocok untuk mencari jalur terpendek dalam graf tak berbobot.

Implementasi ini mengikuti prinsip pemrograman berorientasi objek dan memanfaatkan koleksi data standar Java untuk pengelolaan data yang efektif.

BAB II

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Binary Tree dan Graf merupakan struktur data yang sangat penting dalam pengolahan data. Pada implementasi Binary Tree, berbagai operasi seperti pembentukan pohon, perhitungan jumlah simpul, pencarian data, serta traversal menggunakan metode Inorder, Preorder, dan Postorder dapat dilakukan dengan baik menggunakan teknik rekursif.

Selain itu, pada implementasi struktur data Graf, proses penelusuran simpul dapat dilakukan menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Algoritma DFS melakukan penelusuran hingga ke simpul terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya, sedangkan BFS melakukan penelusuran berdasarkan tingkat kedekatan simpul dengan memanfaatkan struktur data Queue.

Melalui praktikum ini, mahasiswa dapat memahami konsep dasar Binary Tree dan Graf, mengetahui perbedaan cara kerja algoritma traversal yang digunakan, serta mampu mengimplementasikannya menggunakan bahasa pemrograman Java. Pengetahuan ini menjadi dasar yang penting dalam mempelajari berbagai algoritma dan struktur data yang lebih kompleks pada bidang ilmu komputer.

TUGGAS PRATIKUM

Kode Program

```
1 package pekan9_2511533004;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.*;
6
7 public class Petakampus_2511533004 extends JFrame {
8     static Map<String, ArrayList<String>> graph_3004 = new LinkedHashMap<>();
9     static Map<String, Point> pos_3004 = new HashMap<>();
10
11     // Daftar lokasi di Kampus
12     String[] lokasi_3004 = {
13         "Lobby",
14         "Ruang Dosen",
15         "Perpustakaan",
16         "Laboratorium",
17         "Aula",
18         "Kantin",
19         "Masjid",
20         "Gedung Rektorat",
21         "Parkiran",
22         "Pos Satpas"
23     };
24
25     Set<String> visited_3004 = new LinkedHashSet<>();
26     ArrayList<String> path_3004 = new ArrayList<>();
27     String start_3004 = "";
28     String goal_3004 = "";
29     String method_3004 = "";
30
31     // Komponen GUI
32     JComboBox<String> cbStart_3004;
33     JComboBox<String> cbGoal_3004;
34     JComboBox<String> cbMethod_3004;
35     JTextArea hasil_3004;
36     JPanel panelGambar_3004 = new JPanel() {
37         @Override
38         protected void paintComponent(Graphics g) {
39             super.paintComponent(g);
40             drawGraph(g);
41         }
42     };
43
44     public Petakampus_2511533004() {
45         setTitle("Petakampus - 2511533004");
46         setSize(800, 550);
47         setDefaultCloseOperation(EXIT_ON_CLOSE);
48         setLocationRelativeTo(null);
49         setLayout(null);
50         setVisible(true);
51
52         // Inisialisasi Graf dan Posisi Lokasi
53         buatGraph();
54         aturPosisiLokasi();
55
56         // Label
57         JLabel lblStart = new JLabel("Lokasi Awal");
58         lblStart.setBounds(20, 20, 100, 25);
59         add(lblStart);
60
61         JLabel lblGoal = new JLabel("Lokasi Tujuan");
62         lblGoal.setBounds(20, 30, 100, 25);
63         add(lblGoal);
64
65         JLabel lblMethod = new JLabel("Metode Pencarian");
66         lblMethod.setBounds(20, 80, 100, 25);
67         add(lblMethod);
68
69         // Combo Box
70         cbStart_3004 = new JComboBox<>(lokasi_3004);
71         cbStart_3004.setBounds(120, 20, 150, 25);
72         add(cbStart_3004);
73
74         cbGoal_3004 = new JComboBox<>(lokasi_3004);
```

```

75     cbGoal_3004.setBounds(120, 50, 150, 25);
76     add(cbGoal_3004);
77
78     cbMethod_3004 = new JComboBox<>(new String[]{"DPS", "BFS"});
79     cbMethod_3004.setBounds(120, 80, 150, 25);
80     add(cbMethod_3004);
81
82     // Jarak
83     JButton btnCari = new JButton("Cari Jalur");
84     btnCari.setBounds(380, 50, 120, 30);
85     btnCari.addActionListener(e -> cariJalur());
86     add(btnCari);
87
88     // Area Hasil
89     hasil_3004 = new JTextArea();
90     hasil_3004.setBounds(20, 120, 740, 130);
91     hasil_3004.setFont(new Font("Monospaced", Font.PLAIN, 14));
92     hasil_3004.setEditable(false);
93     add(hasil_3004);
94
95     // Panel Gambar Bata
96     panelGambar_3004.setBounds(20, 250, 740, 350);
97     panelGambar_3004.setBackground(Color.WHITE);
98     add(panelGambar_3004);
99 }
100
101 // Menambahkan antar lokasi (Buat Titled Graph)
102 void buatGraph() {
103     for (String s : lokasi_3004) {
104         graph_3004.put(s, new ArrayList<>());
105     }
106     tambahEdge("Lobby", "Pos Setpas");
107     tambahEdge("Lobby", "Parkiran");
108     tambahEdge("Lobby", "Ruang Dosen");
109     tambahEdge("Ruang Dosen", "Perpustakaan");
110     tambahEdge("Ruang Dosen", "Laboratorium");
111     tambahEdge("Laboratorium", "Aula");
112     tambahEdge("Aula", "Kantin");
113     tambahEdge("Kantin", "Masjid");
114     tambahEdge("Masjid", "Gedung Rektorat");
115     tambahEdge("Gedung Rektorat", "Parkiran");
116 }
117
118 void tambahEdge(String a, String b) {
119     graph_3004.get(a).add(b);
120     graph_3004.get(b).add(a);
121 }
122
123 // Menentukan koordinat tampilan untuk setiap lokasi
124 void aturPosisiLokasi() {
125     pos_3004.put("Pos Setpas", new Point(50, 175));
126     pos_3004.put("Lobby", new Point(120, 180));
127     pos_3004.put("Parkiran", new Point(280, 200));
128     pos_3004.put("Ruang Dosen", new Point(280, 50));
129     pos_3004.put("Perpustakaan", new Point(380, 30));
130     pos_3004.put("Laboratorium", new Point(380, 70));
131     pos_3004.put("Aula", new Point(480, 50));
132     pos_3004.put("Kantin", new Point(580, 180));
133     pos_3004.put("Masjid", new Point(680, 150));
134     pos_3004.put("Gedung Rektorat", new Point(780, 200));
135 }
136
137 // Logika Utama Pemrosesan Jalur
138 void cariJalur() {
139     visited_3004.clear();
140     path_3004.clear();
141     hasil_3004.setText("");
142     start_3004 = (String) cbStart_3004.getSelectedItem();
143     goal_3004 = (String) cbGoal_3004.getSelectedItem();
144     method_3004 = (String) cbMethod_3004.getSelectedItem();
145
146     if (start_3004.equals(goal_3004)) {
147         hasil_3004.setText("Lokasi awal dan tujuan sama.");
148         panelGambar_3004.repaint();
149     }

```

```

150         return;
151     }
152
153     long waktuMulai = System.nanoTime();
154     boolean ketemu = false;
155
156     if (method_3004.equals("DFS")) {
157         ketemu = dfs(start_3004);
158     } else if (method_3004.equals("BFS")) {
159         ketemu = bfs(start_3004);
160     }
161
162     long waktuSelesai = System.nanoTime();
163     long lama = (waktuSelesai - waktuMulai) / 1000000; // ms
164
165     if (ketemu) {
166         Collections.reverse(path_3004);
167         hasil_3004.append("== HASIL PENCAIRAN JALIM ==\n");
168         hasil_3004.append("Dari: " + start_3004 + " Ke: " + goal_3004 + "\n");
169         hasil_3004.append("Metode: " + method_3004 + "\n");
170         hasil_3004.append("Waktu Proses: " + lama + " ms\n");
171         hasil_3004.append("Jalur Ditemukan: " + String.join(" ", path_3004) + "\n");
172         hasil_3004.append("Jumlah Langkah: " + (path_3004.size() - 1));
173     } else {
174         hasil_3004.setText("Jalur tidak ditemukan.");
175     }
176     panelGambar_3004.repaint();
177 }
178
179 // Algoritma DFS
180 boolean dfs(String current) {
181     visited_3004.add(current);
182     if (current.equals(goal_3004)) {
183         path_3004.add(current);
184         return true;
185     }
186
187     if (!visited_3004.contains(next)) {
188         if (dfs(next)) {
189             path_3004.add(current);
190             return true;
191         }
192     }
193 }
194 return false;
195 }
196
197 // Algoritma BFS
198 boolean bfs(String start) {
199     Queue<String> antrian = new LinkedList<>();
200     Map<String, String> induk = new HashMap<>();
201     visited_3004.add(start);
202     antrian.add(start);
203
204     while (!antrian.isEmpty()) {
205         String sekarang = antrian.poll();
206         if (sekarang.equals(goal_3004)) {
207             // Mengetahui Jalur
208             String jejak = sekarang;
209             while (jejak != null) {
210                 path_3004.add(jejak);
211                 jejak = induk.get(jejak);
212             }
213             return true;
214         }
215         for (String tetangga : graph_3004.get(sekarang)) {
216             if (!visited_3004.contains(tetangga)) {
217                 visited_3004.add(tetangga);
218                 induk.put(tetangga, sekarang);
219                 antrian.add(tetangga);
220             }
221         }
222     }

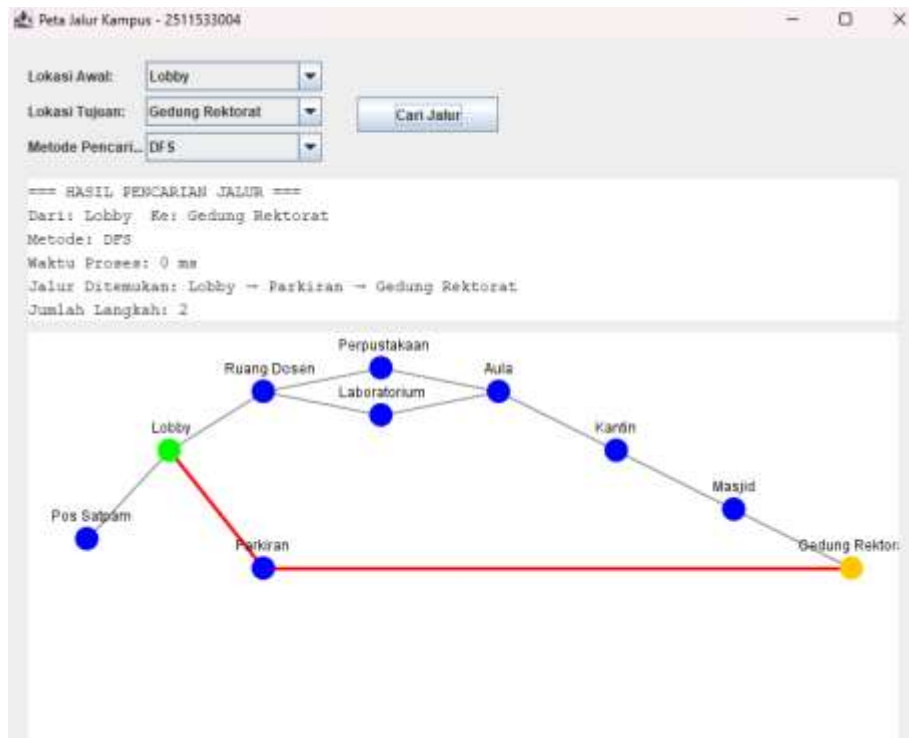
```

```

222     }
223     return false;
224 }
225
226 // Menampilkan Grafik Peta dan Jalur
227 void drawGraph(Graphics g) {
228     Graphics2D g2 = (Graphics2D) g;
229     g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
230
231     // Gambar Garis Tepi
232     g2.setColor(Color.GRAY);
233     g2.setStroke(new BasicStroke(1));
234     for (String asal : graph_3004.keySet()) {
235         Point p1 = pos_3004.get(asal);
236         for (String tujuan : graph_3004.get(asal)) {
237             Point p2 = pos_3004.get(tujuan);
238             if (p1 != null && p2 != null) {
239                 g2.drawLine(p1.x, p1.y, p2.x, p2.y);
240             }
241         }
242     }
243
244     // Gambar Jalur Hasil Pencarian
245     if (!path_3004.isEmpty()) {
246         g2.setColor(Color.RED);
247         g2.setStroke(new BasicStroke(3));
248         for (int i = 0; i < path_3004.size() - 1; i++) {
249             Point p1 = pos_3004.get(path_3004.get(i));
250             Point p2 = pos_3004.get(path_3004.get(i + 1));
251             if (p1 != null && p2 != null) {
252                 g2.drawLine(p1.x, p1.y, p2.x, p2.y);
253             }
254         }
255     }
256
257     // Gambar Titik Lokasi & Nama
258     for (String lok : pos_3004.keySet()) {
259         Point p = pos_3004.get(lok);
260         if (lok.equals(start_3004)) g2.setColor(Color.GREEN);
261         else if (lok.equals(goal_3004)) g2.setColor(Color.ORANGE);
262         else g2.setColor(Color.BLUE);
263
264         g2.fillOval(p.x - 10, p.y - 10, 20, 20);
265         g2.setColor(Color.BLACK);
266         g2.drawString(lok, p.x - (lok.length() * 3), p.y - 15);
267     }
268 }
269
270 public static void main(String[] args) {
271     SwingUtilities.invokeLater(new Runnable() {
272         @Override
273         public void run() {

```

Output



Penjelasan

Program ini adalah aplikasi GUI berbasis Java Swing yang memvisualisasikan peta lokasi di dalam kampus dan mencari jalur terhubung menggunakan metode **DFS (Depth First Search)** dan **BFS (Breadth First Search)**.

Struktur Lokasi & Hubungan

Sistem ini memuat lokasi berikut:

Pos Satpam, Lobby, Parkiran, Ruang Dosen, Perpustakaan, Laboratorium, Aula, Kantin, Masjid, Gedung Rektorat

Hubungan jalur yang dibuat:

- Lobby ↔ Pos Satpam, Parkiran, Ruang Dosen
- Ruang Dosen ↔ Perpustakaan, Laboratorium
- Perpustakaan ↔ Aula
- Laboratorium ↔ Aula
- Aula ↔ Kantin
- Kantin ↔ Masjid
- Masjid ↔ Gedung Rektorat
- Gedung Rektorat ↔ Parkiran

Contoh Hasil Keluaran

Contoh 1: Pencarian DFS

Lokasi Awal: Lobby

Lokasi Tujuan: Masjid

Metode: DFS

Teks Output:

text

=== HASIL PENCARIAN JALUR ===

Dari: Lobby Ke: Masjid

Metode: DFS

Waktu Proses: 1 ms

Jalur Ditemukan: Lobby → Ruang Dosen → Perpustakaan → Aula → Kantin
→ Masjid

Jumlah Langkah: 5

Visualisasi: Jalur garis merah mengikuti urutan pencarian mendalam (selalu bergerak ke cabang pertama sampai habis).

Contoh 2: Pencarian BFS

Lokasi Awal: Lobby

Lokasi Tujuan: Masjid

Metode: BFS

Teks Output:

text

=== HASIL PENCARIAN JALUR ===

Dari: Lobby Ke: Masjid

Metode: BFS

Waktu Proses: 2 ms

Jalur Ditemukan: Lobby → Parkiran → Gedung Rektorat → Masjid

Jumlah Langkah: 3

Visualisasi: Jalur garis merah mengambil jalur terpendek karena BFS memeriksa semua tetangga di tingkat yang sama sebelum masuk lebih dalam.

Tampilan Antarmuka

Saat program dijalankan akan muncul jendela dengan elemen:

1. **Pilihan Lokasi:** Dua kotak dropdown untuk memilih titik awal dan akhir.
2. **Pilihan Metode:** Memilih antara DFS atau BFS.
3. **Tombol Cari:** Memulai proses pencarian.
4. **Panel Hasil:** Menampilkan teks jalur, waktu, dan jumlah langkah.
5. **Peta Visual:**
 - Titik **Hijau**: Lokasi awal.
 - Titik **Oranye**: Lokasi tujuan.
 - Titik **Biru**: Lokasi lain.
 - Garis **Abu-abu**: Jalur umum kampus.
 - Garis **Merah**: Jalur yang dipilih/ditemukan oleh algoritma.

